Università degli Studi Roma Tre
Dipartimento di Ingegneria Civile, Informatica e delle Tecnologie Aeronautiche

Corso di Deep Learning

Docente: Prof. Fabio Gasparetti

Project 1.2

Structured Information Extraction

Report del progetto

Autori

Alfredo Carta
Juan Miguel Pinos Seco

Repository del progetto:
`https://github.com/JuanMiPS/Structured-Information-Extraction/tree/main`

Anno Accademico 2025–2026

Sommario

Questo progetto affronta il task di *structured information extraction*: data una descrizione testuale in linguaggio naturale, il modello deve produrre un output in formato strutturato. L'obiettivo principale è garantire (i) **validità sintattica dell'output JSON prodotto**, (ii) **aderenza rigorosa allo schema** richiesto e (iii) **riduzione delle allucinazioni** nei campi estratti.

L'esperimento utilizza il dataset pubblico **RecipeNLG** e due Small Language Models: **Qwen2.5-3B-Instruct** e **Mistral-7B-Instruct-v0.3**. Dopo una valutazione iniziale in modalità **zero-shot**, i modelli vengono sottoposti a **fine-tuning leggero tramite QLoRA su modelli caricati in quantizzazione 4-bit**.

Le prestazioni vengono valutate tramite metriche automatiche che misurano sia la correttezza sintattica sia la qualità semantica delle informazioni estratte. I risultati mostrano un miglioramento delle prestazioni dopo il fine-tuning, dimostrando l'efficacia di tecniche di adattamento efficienti anche su modelli di dimensioni relativamente ridotte.

Indice

1 Requisiti del progetto e corrispondenza con l'implementazione **2**

2 Workflow sperimentale **2**

3 Task e schema JSON **2**

3.1 Definizione del problema 2

3.2 Schema JSON target 3

4 Dataset **3**

5 Modelli utilizzati **3**

6 Fine-tuning con QLoRA **3**

7 Metriche **4**

8 Risultati **4**

9 Limitazioni **5**

10 Conclusioni **5**

1 Requisiti del progetto e corrispondenza con l'implementazione

Il progetto richiede di:

- tradurre testo naturale in formato strutturato (JSON consigliato);
- curare: **validità sintattica**, **aderenza allo schema**, **riduzione delle allucinazioni**;
- seguire la metodologia: **dataset** → **benchmark** → **2/3 SLM** → risultati **PRE** → **fine-tuning** → risultati **POST**.

L'intera implementazione del progetto è disponibile nel repository:

`https://github.com/JuanMiPS/Structured-Information-Extraction/tree/main`

Nel repository sono presenti:

- il notebook con l'implementazione completa della pipeline sperimentale;
- il codice per la valutazione e la generazione delle metriche;
- i file di output generati dagli esperimenti.

2 Workflow sperimentale

La pipeline sperimentale seguita nel progetto è la seguente:

- Selezione del dataset (**RecipeNLG**)
- Definizione del prompt e dello schema JSON target
- Valutazione iniziale dei modelli in modalità zero-shot (PRE evaluation)
- Fine-tuning leggero dei modelli tramite QLoRA
- Valutazione finale dopo il fine-tuning (POST evaluation)

La pipeline prevede quindi una valutazione iniziale dei modelli senza addestramento specifico (zero-shot), seguita da un fine-tuning leggero tramite QLoRA e da una successiva valutazione comparativa.

3 Task e schema JSON

3.1 Definizione del problema

Dato un input testuale che descrive una ricetta, il modello deve produrre un JSON con i seguenti campi:

- `ingredients`: lista di ingredienti;
- `cooking_time`: tempo di cottura in minuti oppure `null`;
- `temperature`: oggetto contenente valore e unità di temperatura oppure `null`.

3.2 Schema JSON target

Listing 1: Schema JSON atteso

```
{
  "ingredients": ["string", "..."],
  "cooking_time": 123,
  "temperature": {"value": 180, "unit": "C"}
}
```

4 Dataset

È stato utilizzato il dataset pubblico **RecipeNLG**, che contiene descrizioni testuali di ricette.

RecipeNLG è un dataset pubblico progettato per task di generazione e di estrazione di informazioni da ricette. Il dataset include descrizioni testuali di ingredienti, istruzioni di preparazione e metadati culinari, rendendolo adatto a esperimenti di structured information extraction.

Per rendere l'esperimento eseguibile su hardware limitato (Google Colab con GPU Tesla T4), è stato utilizzato un **subset del dataset**:

- Train: 300 esempi
- Test: 40 esempi

Lo split è deterministico tramite seed fissato per garantire la riproducibilità degli esperimenti.

5 Modelli utilizzati

Sono stati utilizzati due Small Language Models:

- **Qwen2.5-3B-Instruct**
- **Mistral-7B-Instruct-v0.3**

Entrambi i modelli sono stati caricati in quantizzazione **4-bit** tramite `bitsandbytes` per ridurre l'uso di memoria GPU.

6 Fine-tuning con QLoRA

Il fine-tuning è stato eseguito tramite la tecnica **QLoRA**, che consente di addestrare modelli di grandi dimensioni aggiornando solo piccoli adapter LoRA.

Configurazione LoRA:

- rank $r = 8$
- alpha = 16
- dropout = 0.05

L'addestramento è stato eseguito con **un numero limitato di step** per contenere il tempo di addestramento e mantenere l'esperimento compatibile con risorse computazionali limitate.

7 Metriche

Le metriche utilizzate sono:

- `valid_json_rate`
- `schema_ok_rate`
- `ingredient_f1`
- `cooking_time_accuracy`
- `temperature_accuracy`

Queste metriche permettono di valutare sia la correttezza sintattica dell'output JSON sia la correttezza semantica delle informazioni estratte.

8 Risultati

Modello	**valid JSON**	**schema OK**	**ingredient F1**	**time acc**	**temp acc**
Qwen 3B (PRE)	1.000	1.000	0.7159	0.600	0.575
Qwen 3B (POST)	1.000	1.000	0.9583	0.650	0.925
Mistral 7B (PRE)	1.000	1.000	0.9079	0.500	0.675
Mistral 7B (POST)	0.975	0.975	0.9750	0.875	0.925

Tabella 1: Confronto delle prestazioni PRE vs POST fine-tuning sul set di valutazione (40 esempi).

I risultati mostrano un miglioramento significativo delle metriche semantiche dopo il fine-tuning per entrambi i modelli. In particolare, si osserva un aumento marcato dell'*ingredient F1*, che misura la correttezza dell'estrazione degli ingredienti, indicando che i modelli riescono a identificare con maggiore precisione le informazioni rilevanti nel testo. Anche le metriche relative agli attributi strutturati (*time accuracy* e *temperature accuracy*) migliorano sensibilmente dopo l'addestramento specifico sul task.

Il modello Mistral 7B mostra prestazioni iniziali più elevate nella fase PRE, probabilmente grazie alla maggiore dimensione del modello e alla qualità dell'addestramento di base. Tuttavia, anche il modello Qwen 3B beneficia in modo significativo del fine-tuning, mostrando un incremento particolarmente evidente nella metrica *ingredient F1*. Questo suggerisce che modelli di dimensioni più ridotte possono adattarsi efficacemente a compiti specifici tramite tecniche di fine-tuning leggere come QLoRA.

Per quanto riguarda la validità sintattica, entrambi i modelli mantengono valori molto elevati nelle metriche *valid JSON* e *schema OK*, indicando che sono generalmente in grado di rispettare il formato strutturato richiesto. Si osserva una lieve diminuzione per Mistral 7B nella configurazione POST, che potrebbe essere dovuta a una maggiore variabilità generativa dopo l'addestramento. Nel complesso, i risultati confermano che il fine-tuning consente di migliorare significativamente la qualità dell'estrazione strutturata mantenendo al contempo una buona affidabilità nella generazione del formato JSON.

9 Limitazioni

Il dataset utilizzato per il fine-tuning è relativamente piccolo (300 esempi di training), e questo limita la capacità dei modelli di apprendere pattern più complessi.

Inoltre le etichette utilizzate derivano da un processo di parsing automatico delle ricette e non da annotazioni manuali, il che può introdurre rumore nei dati.

Esperimenti futuri potrebbero utilizzare dataset più ampi o annotazioni manuali per migliorare ulteriormente le prestazioni.

10 Conclusioni

Il progetto dimostra come tecniche di fine-tuning efficienti come **QLoRA** possano migliorare significativamente le prestazioni dei Small Language Models su task di *structured information extraction*.

Nonostante l'utilizzo di un dataset di dimensioni ridotte, entrambi i modelli mostrano miglioramenti evidenti nelle metriche di estrazione semantica dopo il fine-tuning.

Questi risultati evidenziano come tecniche di adattamento efficienti possano essere sufficienti per specializzare modelli generativi su compiti strutturati specifici, anche utilizzando risorse computazionali limitate.